

Name: Ayushi Shringi
SID4: 0036
DOMAIN_ID: 4
Domain Entity: Open-source package vulnerabilities
Port: 8036
Prefix: s0036
SEED: 0036
VERIFY_SEED: 260036
Python Version: Python 3.12.14
Embedding Model: sentence-transformers/all-MiniLM-L6-v2
Hardware: MacBook Pro
Chip: Apple M5 Pro
Memory: 24 GB
Operating System: macOS Tahoe 26.5.1
GitHub Repository: <https://github.com/ayushishringi/data260-0036/tree/hw3> Code
Commit Hash: e4c18aa443fbfd04018efe8e3a2d0ff19530c37b
Git Tag:

Part 1 - FastAPI Authentication and Session Management

1. Routes

Route	Method	Purpose
/	GET	Displays the welcome page and navigation links
/login	GET	Displays the login form
/login	POST	Validates credentials and creates a session
/dashboard	GET	Protected page showing the logged-in user
/logout	GET	Clears the session and redirects to the home page

```
def get_current_user(request: Request):
    user = request.session.get("user")
    last_seen = request.session.get("last_seen")

    if user is None or last_seen is None:
        request.session.clear()
        return None

    if time.time() - float(last_seen) > IDLE_TIMEOUT_SECONDS:
        request.session.clear()
        return None

    request.session["last_seen"] = time.time()
    return user


@router.get("/", response_class=HTMLResponse)
async def home(request: Request):
    return templates.TemplateResponse(
        request=request,
        name="home.html",
        context={
            "user": get_current_user(request),
        },
    )


@router.get("/login", response_class=HTMLResponse)
async def login_page(request: Request):
    return templates.TemplateResponse(
        request=request,
        name="login.html",
        context={
            "user": get_current_user(request),
            "error": None,
        },
    )
```

```

3
4 @router.get("/login", response_class=HTMLResponse)
5 async def login_page(request: Request):
6     return templates.TemplateResponse(
7         request=request,
8         name="login.html",
9         context={
10             "user": get_current_user(request),
11             "error": None,
12         },
13     )
14
15
16 @router.post("/login", response_class=HTMLResponse)
17 async def login(
18     request: Request,
19     username: str = Form(...),
20     password: str = Form(...),
21 ):
22     account = USERS.get(username.strip().lower())
23
24     if account is None or account["password"] != password:
25         return templates.TemplateResponse(
26             request=request,
27             name="login.html",
28             context={
29                 "user": None,
30                 "error": "Invalid username or password.",
31             },
32             status_code=401,
33         )
34
35     request.session["user"] = {
36         "username": username.strip().lower(),
37         "name": account["name"],
38     }
39
40     request.session["last_seen"] = time.time()
41
42     return RedirectResponse("/dashboard", status_code=303)
43
44
45 @router.get("/dashboard", response_class=HTMLResponse)
46 async def dashboard(request: Request):
47     # ... (code continues)

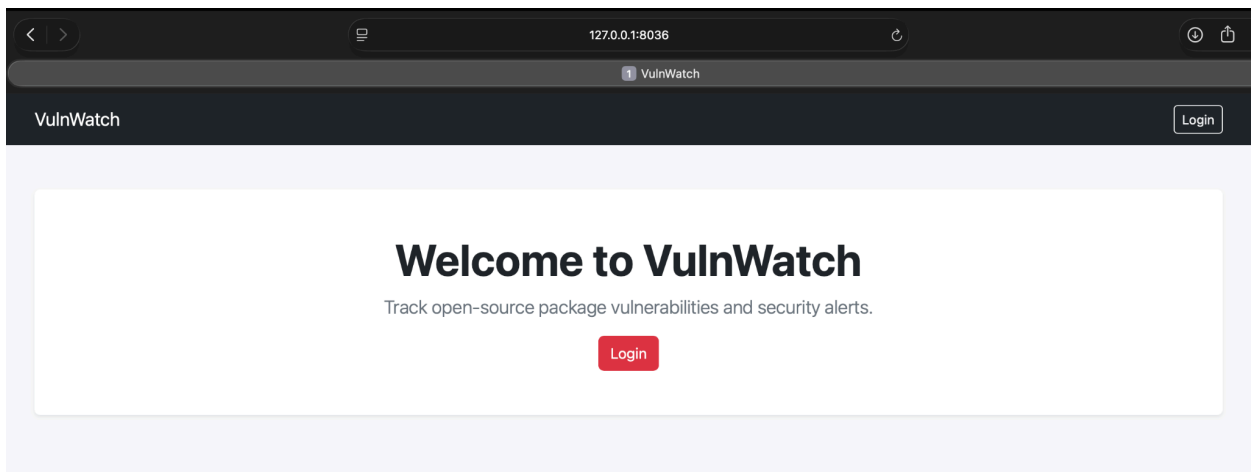
```

```

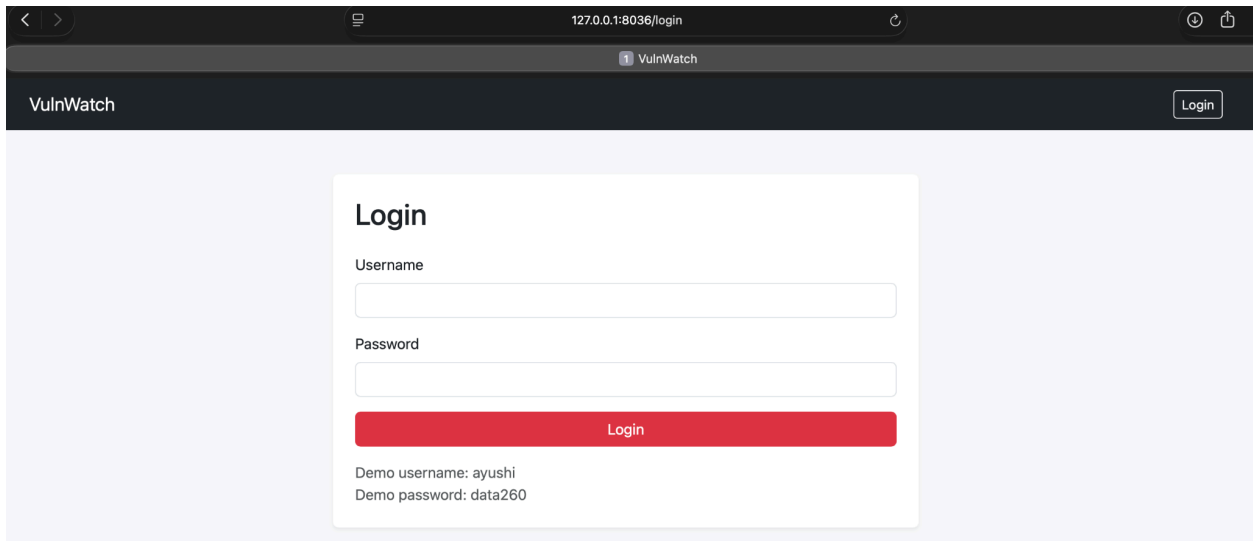
94
95 @router.get("/dashboard", response_class=HTMLResponse)
96 async def dashboard(request: Request):
97     user = get_current_user(request)
98
99     if user is None:
100         return RedirectResponse("/login", status_code=303)
101
102     return templates.TemplateResponse(
103         request=request,
104         name="dashboard.html",
105         context={
106             "user": user,
107         },
108     )
109
110
111 @router.get("/logout")
112 async def logout(request: Request):
113     request.session.clear()
114     return RedirectResponse("/", status_code=303)

```

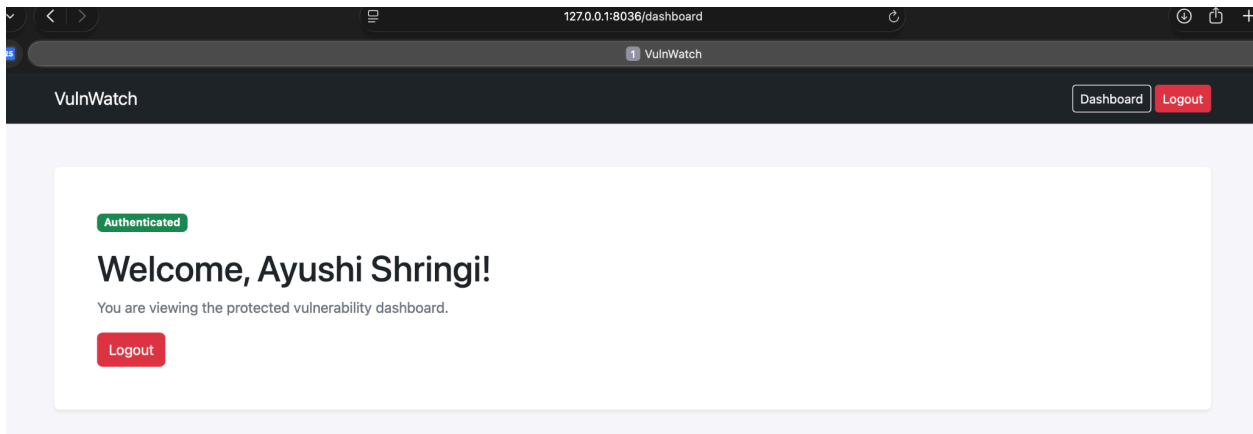
Home Page -



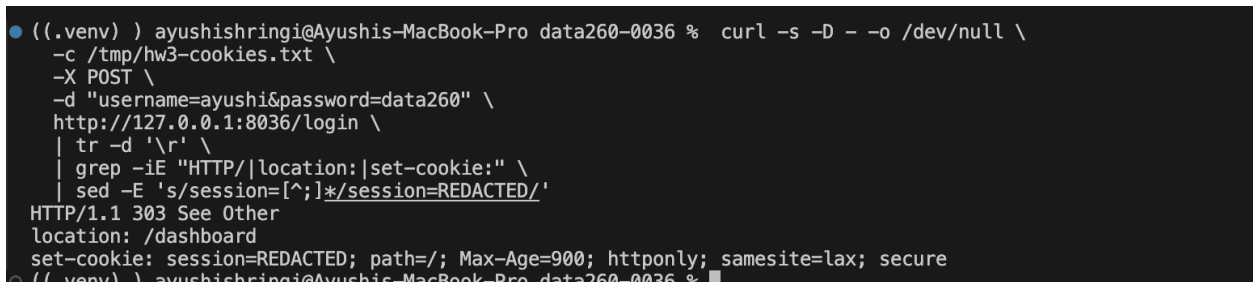
Login Page -



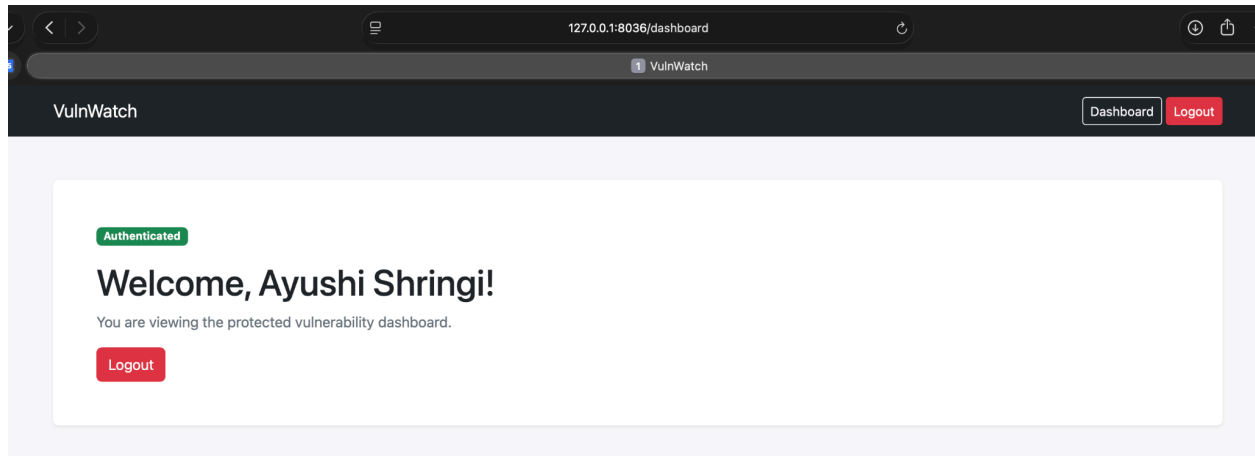
Successful Login -



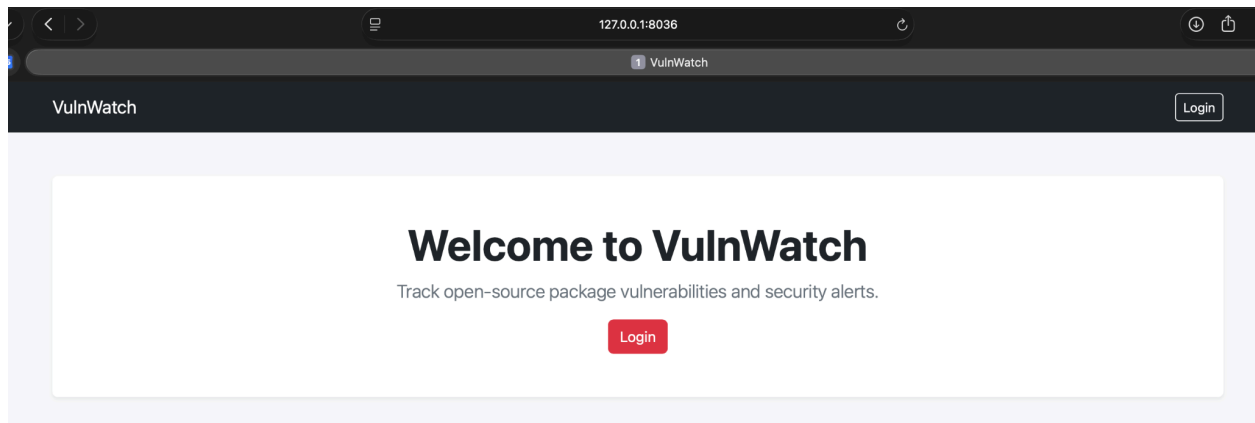
Secure Session Cookie -



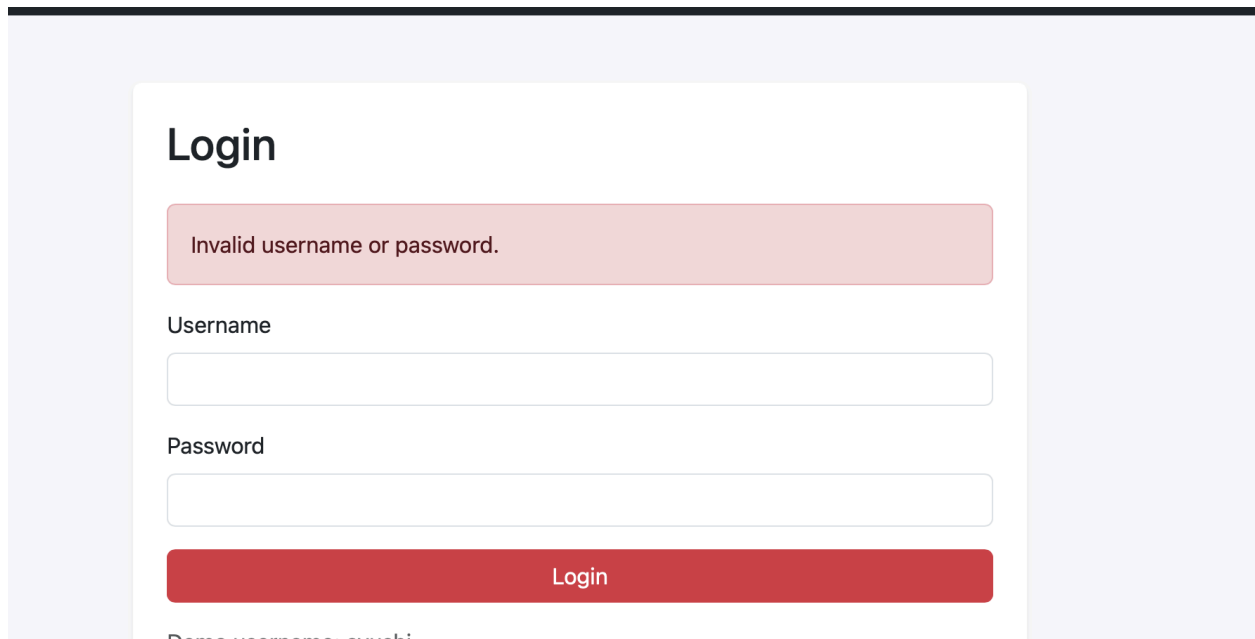
Home Page -



Home Page for Logged out user -



Incorrect Creds -



Templates for authentication page -

```
1 <!DOCTYPE html>
2 <html lang="en">
3 <head>
4   <meta charset="UTF-8">
5   <meta name="viewport" content="width=device-width, initial-scale=1">
6
7   <title>{{ title or "VulnWatch" }}</title>
8
9   <link
10     href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.3/dist/css/bootstrap.min.css"
11     rel="stylesheet"
12   >
13 </head>
14
15 <body class="bg-light">
16
17 <nav class="navbar navbar-dark bg-dark">
18   <div class="container">
19     <a class="navbar-brand" href="/">VulnWatch</a>
20
21     <div>
22       {% if user %}
23         <a class="btn btn-outline-light btn-sm" href="/dashboard">
24           Dashboard
25         </a>
26         <a class="btn btn-danger btn-sm" href="/logout">
27           Logout
28         </a>
29       {% else %}
30         <a class="btn btn-outline-light btn-sm" href="/login">
31           Login
32         </a>
33       {% endif %}
34     </div>
35   </div>
36 </nav>
37
38 <main class="container py-5">
39   {% block content %}{% endblock %}
40 </main>
41
42 </body>
```

templates > <> dashboard.html > ...

```
1  {% extends "base.html" %}
2
3  {% block content %}
4
5  <div class="card shadow-sm border-0">
6      <div class="card-body p-5">
7
8          <span class="badge bg-success mb-3">
9              Authenticated
10         </span>
11
12         <h1>
13             Welcome, {{ user.name }}!
14         </h1>
15
16         <p class="text-secondary">
17             You are viewing the protected vulnerability dashboard.
18         </p>
19
20         <a href="/logout" class="btn btn-danger">
21             Logout
22         </a>
23     </div>
24 </div>
25
26
27 {% endblock %}
```

templates > <> home.html > ...

```
1  {% extends "base.html" %}
2
3  {% block content %}
4
5      <div class="card shadow-sm border-0">
6          <div class="card-body p-5 text-center">
7              <h1 class="display-5 fw-bold">
8                  Welcome to VulnWatch
9              </h1>
10
11              <p class="lead text-secondary">
12                  Track open-source package vulnerabilities and security alerts.
13              </p>
14
15              {% if user %}
16                  <a href="/dashboard" class="btn btn-danger">
17                      Go to Dashboard
18                  </a>
19              {% else %}
20                  <a href="/login" class="btn btn-danger">
21                      Login
22                  </a>
23              {% endif %}
24          </div>
25      </div>
26
27  {% endblock %}
```

```

3  {% block content %}
4
5  <div class="row justify-content-center">
6      <div class="col-md-6">
7
8          <div class="card shadow-sm border-0">
9              <div class="card-body p-4">
10
11                  <h2 class="mb-4">Login</h2>
12
13                  {% if error %}
14                      <div class="alert alert-danger">
15                          {{ error }}
16                      </div>
17                  {% endif %}
18
19                  <form method="post" action="/login">
20
21                      <div class="mb-3">
22                          <label for="username" class="form-label">
23                              Username
24                          </label>
25
26                          <input
27                              id="username"
28                              name="username"
29                              class="form-control"
30                              required
31                          >
32                      </div>
33
34                      <div class="mb-3">
35                          <label for="password" class="form-label">
36                              Password
37                          </label>
38
39                          <input
40                              id="password"
41                              name="password"
42                              type="password"
43                              class="form-control"
44                              required
45                          >
46                      </div>
47
48                      <button type="submit" class="btn btn-danger w-100">
49                          Login
50                      </button>
51                  </form>
52

```

Expired session redirected to login -

```
((.venv) ) ayushishringi@Ayushis-MacBook-Pro data260-0036 % curl -i -b /tmp/hw3-timeout.txt \
http://127.0.0.1:8036/dashboard
HTTP/1.1 303 See Other
date: Mon, 21 Sep 2026 18:25:59 GMT
server: uvicorn
content-length: 0
location: /login
vary: Cookie
set-cookie: session=null; path=/; expires=Thu, 01 Jan 1970 00:00:00 GMT; httponly; samesite=lax

((.venv) ) ayushishringi@Ayushis-MacBook-Pro data260-0036 %
```

Three LlamaIndex chunking techniques

```

c > retrieval.py > query_indexes
9 |     TokenTextSplitter,
10 | )
11 | from llama_index.embeddings.huggingface import HuggingFaceEmbedding
12 |
13 |
14 | ROOT = Path(__file__).resolve().parents[1]
15 | CORPUS_FILE = ROOT / "corpus" / "github_reviewed_advisories_snapshot.txt"
16 |
17 | MODEL_NAME = "sentence-transformers/all-MiniLM-L6-v2"
18 |
19 |
20 | def load_corpus() -> list[Document]:
21 |     text = CORPUS_FILE.read_text(encoding="utf-8")
22 |
23 |     if len(text.encode("utf-8")) < 200_000:
24 |         raise ValueError("Corpus must be at least 200 KB.")
25 |
26 |     return [
27 |         Document(
28 |             text=text,
29 |             metadata={
30 |                 "source": "GitHub Advisory Database",
31 |                 "corpus_file": str(CORPUS_FILE),
32 |             },
33 |         )
34 |     ]
35 |
36 |
37 | def average_chunk_length(nodes) -> float:
38 |     if not nodes:
39 |         return 0.0
40 |
41 |     lengths = [
42 |         len(node.get_content())
43 |         for node in nodes
44 |     ]
45 |
46 |     return round(float(np.mean(lengths)), 2)
47 |
48 |
49 | def create_pipeline(nodes, embed_model):
50 |     return {
51 |         "index": VectorStoreIndex(
52 |             nodes,
53 |             embed_model=embed_model,
54 |         ),

```

```

49 def create_pipeline(nodes, embed_model):
50     nodes : nodes,
51     "embed_model": embed_model,
52     "chunk_count": len(nodes),
53     "avg_chunk_length": average_chunk_length(nodes),
54 }
55
56
57
58
59
60
61
62 def build_indexes():
63     embed_model = HuggingFaceEmbedding(
64         model_name=MODEL_NAME
65     )
66
67     Settings.embed_model = embed_model
68
69     documents = load_corpus()
70
71     token_parser = TokenTextSplitter(
72         chunk_size=512,
73         chunk_overlap=50,
74     )
75
76     token_nodes = token_parser.get_nodes_from_documents(documents)
77
78     semantic_parser = SemanticSplitterNodeParser(
79         buffer_size=1,
80         breakpoint_percentile_threshold=95,
81         embed_model=embed_model,
82     )
83
84     semantic_nodes = semantic_parser.get_nodes_from_documents(documents)
85
86     window_parser = SentenceWindowNodeParser.from_defaults(
87         window_size=3,
88         window_metadata_key="window",
89         original_text_metadata_key="original_text",
90         include_prev_next_rel=True,
91     )
92
93     window_nodes = window_parser.get_nodes_from_documents(documents)
94
95     return {
96         "token": create_pipeline(
97             token_nodes,
98             embed_model,
99         ),

```

```

103         ),
104         "sentence_window": create_pipeline(
105             window_nodes,
106             embed_model,
107         ),
108     }
109
110
111 def cosine_similarity(query_vector, document_vector) -> float:
112     query_norm = np.linalg.norm(query_vector)
113     document_norm = np.linalg.norm(document_vector)
114
115     if query_norm == 0 or document_norm == 0:
116         return 0.0
117
118     return float(
119         np.dot(query_vector, document_vector)
120         / (query_norm * document_norm)
121     )
122
123
124 def query_indexes(indexes, question: str, top_k: int = 3) -> dict:
125     results = {}
126
127     for name, pipeline in indexes.items():
128         embed_model = pipeline["embed_model"]
129
130         query_embedding = np.asarray(
131             embed_model.get_query_embedding(question),
132             dtype=np.float32,
133         )
134
135         retriever = pipeline["index"].as_retriever(
136             similarity_top_k=top_k
137         )
138
139         started = time.perf_counter()
140         source_nodes = retriever.retrieve(question)
141         retrieval_latency_ms = (
142             time.perf_counter() - started
143         ) * 1000
144
145         retrieved_rows = []
146         document_vectors = []
147

```

```

124 def query_indexes(indexes, question: str, top_k: int = 3) -> dict:
153
154     document_embedding = np.asarray(
155         embed_model.get_text_embedding(text),
156         dtype=np.float32,
157     )
158
159     document_vectors.append(document_embedding)
160
161     retrieved_rows.append(
162         {
163             "rank": rank,
164             "store_score": (
165                 float(node_with_score.score)
166                 if node_with_score.score is not None
167                 else None
168             ),
169             "cosine_similarity": round(
170                 cosine_similarity(
171                     query_embedding,
172                     document_embedding,
173                 ),
174                 6,
175             ),
176             "chunk_length": len(text),
177             "preview": text[:160].replace("\n", " "),
178         }
179     )
180
181     if document_vectors:
182         document_matrix = np.vstack(document_vectors)
183         document_shape = list(document_matrix.shape)
184     else:
185         document_shape = [0, int(query_embedding.shape[0])]
186
187     results[name] = {
188         "technique": name,
189         "question": question,
190         "chunk_count": pipeline["chunk_count"],
191         "avg_chunk_length": pipeline["avg_chunk_length"],
192         "query_embedding_dimension": int(
193             query_embedding.shape[0]
194         ),
195         "query_embedding_first_8": [
196             round(float(value), 8)
197             for value in query_embedding[:8]

```

Retrieval outputs and comparison metrics -

```
((.venv) ) ayushishringi@Ayushtis-MacBook-Pro data260-0036 % .venv/bin/python scripts/run_hw3_retrieval.py
Building the three retrieval indexes...
Warning: You are sending unauthenticated requests to the HF Hub. Please set a HF_TOKEN to enable higher rate limits and
faster downloads.
Loading weights: 100%|██████████████████████████████████████████████████████████████████████████████| 103/103 [00:00<00:00, 8664.18it/s]

Question q1: Which packages are affected by prototype pollution vulnerabilities?
token: 3 chunks, latency=18.8241 ms
semantic: 3 chunks, latency=3.6955 ms
sentence_window: 3 chunks, latency=15.2547 ms

Question q2: What versions of Apache Pulsar are affected by security vulnerabilities?
token: 3 chunks, latency=14.1337 ms
semantic: 3 chunks, latency=3.9021 ms
sentence_window: 3 chunks, latency=14.415 ms

Question q3: What are common mitigation steps described in the advisories?
token: 3 chunks, latency=14.026 ms
semantic: 3 chunks, latency=3.7879 ms
sentence_window: 3 chunks, latency=14.4105 ms

Question q4: Which vulnerabilities allow remote code execution?
token: 3 chunks, latency=14.2717 ms
semantic: 3 chunks, latency=4.846 ms
sentence_window: 3 chunks, latency=14.8071 ms

Question q5: Which advisories mention denial of service?
token: 3 chunks, latency=13.8142 ms
semantic: 3 chunks, latency=3.8962 ms
sentence_window: 3 chunks, latency=14.473 ms

Summary metrics:
{
  "token": {
    "questions_evaluated": 5,
    "chunks_produced": 1737,
    "average_chunk_length": 1091.45,
    "top_1_cosine_mean": 0.484747,
    "mean_at_k_cosine": 0.427933,
    "recall_at_k": 0.6,
    "mean_retrieval_latency_ms": 15.01394
  },
  "semantic": {
    "questions_evaluated": 5,
    "chunks_produced": 93,
    "average_chunk_length": 18464.8,
    "top_1_cosine_mean": 0.405813,
    "mean_at_k_cosine": 0.333585,
    "recall_at_k": 0.2,
    "mean_retrieval_latency_ms": 4.02554
  },
  "sentence_window": {
    "questions_evaluated": 5,
```

```

    "mean_retrieval_latency_ms": 4.02554
  },
  "sentence_window": {
    "questions_evaluated": 5,
    "chunks_produced": 1834,
    "average_chunk_length": 936.33,
    "top_1_cosine_mean": 0.534835,
    "mean_at_k_cosine": 0.479562,
    "recall_at_k": 0.4,
    "mean_retrieval_latency_ms": 14.67206
  }
}

```

Retrieval-quality comparison -

```

1  # HW3 Retrieval Metrics
2
3  ## Configuration
4
5  - Domain ID: 4
6  - Domain: Open-source package vulnerabilities
7  - Corpus: `corpus/github_reviewed_advisories_snapshot.txt`
8  - Embedding model: `sentence-transformers/all-MiniLM-L6-v2`
9  - Questions evaluated: 5
10 - Retrieval depth: k = 3
11 - Retrieval mode: Retrieval-only; no external LLM was used
12
13 ## Retrieval Quality Comparison
14
15 | Technique | Chunks | Average chunk length | Top-1 cosine | Mean@3 cosine | Recall@3 | Mean retrieval latency (ms) |
16 |---|---|---|---|---|---|---|
17 | Token | 1737 | 1091.45 | 0.484747 | 0.427933 | 0.60 | 16.586620 |
18 | Semantic | 93 | 18464.80 | 0.405813 | 0.333585 | 0.20 | 4.152180 |
19 | Sentence-window | 1834 | 936.33 | 0.534835 | 0.479562 | 0.40 | 15.155020 |
20
21 ## Interpretation
22
23 Sentence-window chunking produced the strongest cosine-similarity results. It achieved the highest average top-1 cosine score and the highest mean@3 cosine
24 score. Its sentence-level chunks preserved enough local context while remaining focused on individual advisory statements.
25
26 Token chunking achieved the highest Recall@3 using the expected source-match check. It produced moderately sized chunks and performed consistently across
27 the five questions.
28
29 Semantic chunking was the fastest at retrieval time and produced far fewer chunks. However, its average chunk length was much larger, and its cosine and
30 recall values were lower. Large semantic chunks may contain several unrelated advisory sections, which can reduce retrieval precision for focused questions.
31
32 ## Conclusion
33
34 For this vulnerability-advisory corpus, Sentence-window chunking is the best overall technique based on cosine similarity. Token chunking is a strong
35 alternative when source-match recall is prioritized. Semantic chunking is fastest but less precise for these questions because its chunks are very large.
36
37 ## Reproducibility
38
39 The raw per-query and per-technique results are stored in:
40
41 ```text
42 reports/hw03/raw/retrieval_outputs.json

```

Retrieval-Only RAG with Three LlamaIndex Chunking Techniques -

The second part compares three LlamaIndex chunking techniques for retrieval-only RAG over a local corpus of open-source package vulnerability advisories. The corpus

contains 500 GitHub-reviewed advisory records and is 1,717,675 bytes, which exceeds the required 200 KB minimum.

The embedding model used for all three techniques was ``sentence-transformers/all-MiniLM-L6-v2``. Each technique produced an in-memory `VectorStoreIndex`. The retrieval function computed the query embedding, retrieved the top three nodes, calculated cosine similarity with each returned document embedding, recorded chunk lengths, and measured retrieval latency.

1. Chunking Techniques - Token-based chunking used ``TokenTextSplitter`` with a chunk size of 512 tokens and an overlap of 50 tokens. Semantic chunking used ``SemanticSplitterNodeParser`` with a buffer size of 1 and a breakpoint percentile threshold of 95. Sentence-window chunking used ``SentenceWindowNodeParser`` with a three-sentence window and neighboring sentence relationships enabled.

2. Retrieval Results - Sentence-window chunking produced the strongest cosine-similarity results, with a mean top-1 cosine score of 0.534835 and a mean@3 cosine score of 0.479562. Token chunking achieved the highest Recall@3 value of 0.60. Semantic chunking was the fastest technique, with a mean retrieval latency of 4.15218 milliseconds, but it produced very large chunks and had the lowest cosine and recall values.

3. Observations - Sentence-window chunking performed well because it preserved sentence-level meaning while retaining nearby context. This was useful for vulnerability advisories, where the affected package, vulnerability behavior, and mitigation information are often described across neighboring sentences.

Token chunking provided a useful balance between chunk size and retrieval coverage. Semantic chunking created only 93 chunks, but the average chunk length was 18,464.80 characters. These very large chunks sometimes combined unrelated advisory sections, which likely reduced retrieval precision.

4. Incorrect High-Scored Retrieval - At least one high-scored retrieval did not contain the expected answer. For example, the prototype-pollution question returned unrelated vulnerability text for some techniques, including sections about Apache Pulsar and other packages instead of the expected web3-utils advisory. This likely occurred because the embedding model recognized general security and vulnerability language as semantically similar even though the retrieved chunk did not contain the specific package requested.

5. Conclusion - For this open-source package vulnerability corpus, Sentence-window chunking is the best overall technique based on the highest top-1 and mean@3 cosine scores. Token chunking is a strong alternative when recall is the main priority. Semantic chunking is fastest, but its very large chunks reduce precision for focused vulnerability questions.

AI Use Disclosure

I used an AI assistant to understand the HW3 requirements, plan the authentication and retrieval implementation, debug errors, and review the retrieval metrics. I personally edited the files in VS Code, ran the commands, tested the application, verified the outputs, and committed the work.

An initial retrieval implementation incorrectly used LlamaIndex's default OpenAI query engine and failed because no ``OPENAI_API_KEY`` was configured. I identified this from the traceback and changed the implementation to retrieval-only mode using ``as_retriever()``. This removed the external LLM dependency and allowed the application to run locally.